Building Production-Ready Retrieval Systems

This comprehensive and practical guide will help you successfully design, build, and run retrieval-augmented generation (RAG) systems that are viable in production environments.

Retrieval-augmented generation (RAG) combines information retrieval with large language model (LLM) capabilities to provide answers that are based on current, validated sources rather than only on previously accessed information. By using more contextual data added to the answer at the time of response, RAG helps produce 'grounded' (i.e., based on evidence), real-time, valid answers.

LLMs are incredibly useful but have many limitations, the most serious one being that they can generate answers based only on their training data. As a result, they are not suitable for providing accurate responses where the answers require current information, knowledge of a specific domain, or proprietary information.

RAG provides the solution to these problems by integrating two separate systems:

- Retriever that retrieves the relevant data from an external data source.
- Generator (LLM) that generates answers based on the data returned from the retrieved context.

Rather than asking the model to retrieve everything, RAG gives it access to retrieve information as needed while processing requests. Therefore, it is particularly useful in the following areas:

- Enterprise knowledge base
- Product documentations and APIs
- Internal tools (i.e., HR, finance, support)
- Regulated industries (i.e., healthcare, legal, finance)

Most RAG examples found online are demo-grade systems. They work well for tutorials but fail under real-world constraints.

Table 1: The difference between demo RAG and production RAG systems

Dimension	Demo RAG	Production RAG
Data size	Few documents	Millions of chunks
Updates	Manual, static	Continuous ingestion
Retrieval	Simple vector search	Hybrid + re-ranking
Latency	Ignored	Static SLAs
Evaluation	Manual inspections	Automated metrics
Reliability	Best effort	Fault-tolerant

RAG architecture

To create a production-level RAG system, we use a structured pipeline design that separates the knowledge retrieval process from the language generation process. The operating flow of the RAG system has a defined path to follow during runtime. First, the system analyses and rewrites a user query in order to improve the effectiveness of its retrieval process. Next, the system uses a 'retriever' to find matching vector, or hybrid, database entries for the query, returning a set of candidate text 'chunks' (potential pieces of text).

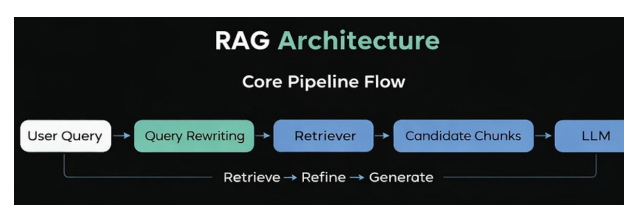

Figure 1: RAG architecture

Finally, the retrieved chunks are refined and assembled into a bounded context which can be provided to the LLM